Preliminary Design Document: MatPotLib

Justin Tong, Ehsan Ayoubi, Amer Samman, Dylan Johnson, Sultan Lodi

Table of Contents

1. Executive Summary

1.1. Project Vision

MatPotLib is a solution to everyone's plant care needs, for people who have great ambitions, but not enough time nor the skill to grow healthy happy plants. It is there to bridge the gap between humans and their botanical friends, by giving a friendly user interface users can see detailed understanding of their plant's condition. This removes any guesswork or chore-heavy data gathering that is otherwise required. By using physical sensors inside of the plant pot, and sending the data to a cloud-based backend, which a front-end mobile can display, users will be empowered to use both real-time and background monitoring of their plants to create more actionable insights.

1.1. System Architecture

This project has multiple layers to its architecture to ensure a simple and robust separation of concerns, which gives users a seamless and reliable experience. Here is a quick overview:

Layer	**Purpose**
Hardware Layer	This layer contains the physical component that communicate plant environment conditions to the cloud server. The components will relay information to an Arduino Nano ESP32, chosen for it's compact form and built-in WiFi capability. Some other hardware includes a soil moisture sensor, climate tracking sensor (temperature, humidity, and pressure), and a light intensity sensor. More information can be found in "Section 5.1 Hardware."
Backend Layer	This layer contains the code to connect all other layers together, from the hardware data collection to the database storage, and then again to the frontend's user display. Built with NestJS and TypeScript, chosen

	primarily because of its robustness in large scale applications and familiarity with certain team members. This layer will validate incoming sensor payloads and manage the logic for generating plant environment recommendations. More information can be found in "Section 5.2 Backend."
Database Layer	Supabase will serve as the primary relational database. Using PostgreSQL as the engine, it allows for managing more complex relational databases between users, devices, plants, and time-series sensor readings. More information on the tables can be found in "Section 5.3 Database."
Frontend Layer	The mobile application will have a frontend built with React Native and Expo, allowing for high performance and seamless experience to users in a cross-platform experience, allowing users to monitor their plants from any location or device. More information can be found in "Section 5.3 Frontend."

1.3. User Experience and Primary Features

The user journey begins with a simple setup process, where the user can create or log into an account. Afterwards, the user can pair a plant pot to their account with Bluetooth or WiFi. Afterwards, the user can select the plant the pot holds, and the app will automatically fetch care requirements for the specific plant. The pot will then begin sending regular environmental metrics at a regular interval automatically. The mobile application provides a view on each of the user's registered pots, visualizing historical trends of their environments. If a pot begins to enter an unideal environment, then the user will be sent a push notification indicating the problem detected. This device will create a more stress-free plant care experience for the user.

- **Proactive Health Alerts:** The system will identify unideal conditions before they cause harm to the plan, sending notifications directly to the user's device of their choice.

- **Species-Specific Intelligence:** Rather than providing generic moisture or light data, the system tailors recommendations to the exact species of the plant. This ensures the app is accessible to users with any plant, and removes potentially dangerous scenarios like treating a succulent the same as a tropical plant.
- **Historical Trend Analysis:** All data collected by sensors will be readily available to the user to view, allowing them to identify historical patterns in their home environment, opening the doors to strategic plant placement based on seasonal changes.

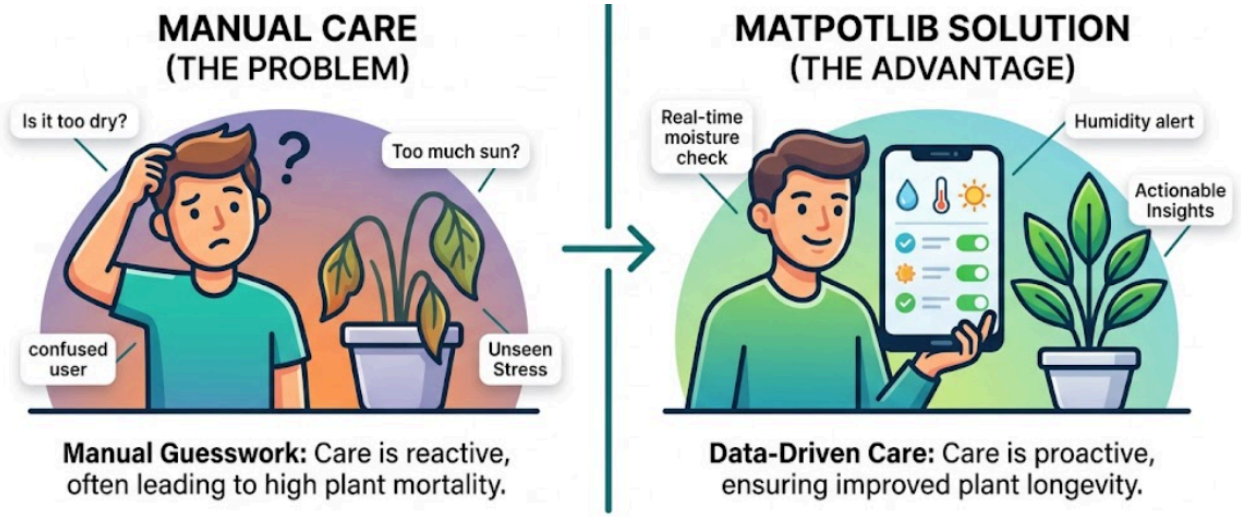

Figure 1: Comparative overview of user interaction models [3].

1.3. Business Value and Outcomes

MatPotLib primarily provides value by increasing survival rates of customers' plants, therefore reducing turnover costs. The system is designed to pay for itself in a decrease in plant rehabilitation and replacement costs, as well as customer time saved. With the saved time from manual and potentially inaccurate environment checks, the outcome is expected to be a more sustainable plant-life ecosystem that also provides customers with better psychological benefits of greenery and plant care, without the associated anxiety of maintenance.

1.4. Project Status and Recommendations

The project currently has individual demo's and POC's available for each component. It's prepared to transition from the design and prototyping phase to a full-scale implementation and

testing phase cycle. Team members will begin integrating components together, and work towards the goal of having a working MVP by the end of the semester.

2. Problem Statement

2.1. Problem Context

Gardening and plant care has become a standard in recent home styling trends, as it gives homes a refreshing and relaxing look. Unfortunately, the majority of plant owners lack experience and time to care for a diverse set of vegetation. Beginners can find themselves in a situation where they are responsible for an organism, without enough data on how to care for it, or what the plant needs. This approach relies on human observation and guesswork, which is an unreliable and error-prone approach for detecting early stages of environmental stress.

2.2. Core Pain Point

The core issue is lack of information early on. For example, a plant can experience root rot due to overwatering, or an extended period of low light, but these conditions can be invisible to the naked eye for many weeks. By the time the user sees the yellowing leaves or wilting stems from the root rot, it is often too late, and can end up with the plant dead. This can sometimes lead to a caregiver feeling helpless and eventually abandoning the hobby entirely.

2.3. Negative Impact and Potential Consequences

There are multiple potential consequences of failing to manually care for plants, which have real and tangible effects:

- **Financial Waste:** Some consumers, especially hobbyists spend hundreds of dollars annually on plants. If these plants were to die due to poor care, they can cause a significant financial burden to constantly replace them.
- **Environmental Impact:** Constant disposal and replacement of plants contribute to unnecessary waste and carbon footprint.

- **Psychological Disruption:** While users often care for plants for the psychological calmness it brings, it can have the opposite effect if users associate plants with feelings of inadequacy and a source of stress.
- **Time Waste:** Manually checking plant environments, like sticking a finger in the soil to check moisture levels, or moving a pot around to find the best spot, are inefficient methods and don't provide any quantifiable results.

3. Proposed Solution

3.1. Solution Introduction

The Smart Plant System is a plant monitoring and care solution designed to help users maintain healthy plants by tracking conditions and providing feedback. Plant conditions are measured and are then transmitted to the team's service. Based on the optimal conditions the plant should have, actionable recommendations are provided to the user through a connected application to improve plant health and longevity. This system helps to reduce the guesswork that happens in plant care, especially when it comes to beginners, by combining real-world sensor data with plant-specific knowledge and data-driven insights tailored to their specific plant and environment. The team also wanted to mention that more advanced features beyond the team's minimum viable product, including, but not limited to, automated watering (autonomous action rather than suggestions for users), trend analysis, predictive insights, and an interactive chatbot, are considered as future additions.

3.2. System Overview

For planning the team's project, the team first wanted to establish the functionality of the finished minimum viable product. The team wanted a system that provided a mobile application that serves as a primary hub for the user to view and organize plant information, as well as receive push notifications for plant care suggestions. In the app, the user will be able to configure their specific plants by selecting them from the team's database or setting preferences manually. The application will also provide details on logged condition data that the user can analyze. To deliver this, the team first needed a place to source the information to base the app's suggestions

on. This will come from physical sensor hardware that the user must attach to their plant pot and pair with the app. Once set up, the hardware will operate autonomously, reading important metrics and sending this data to the backend system to assess the plant's health. By comparing these readings to the specific plant's optimal conditions, the system will create actionable recommendations if conditions are suboptimal before sending them as alerts to the user's app.

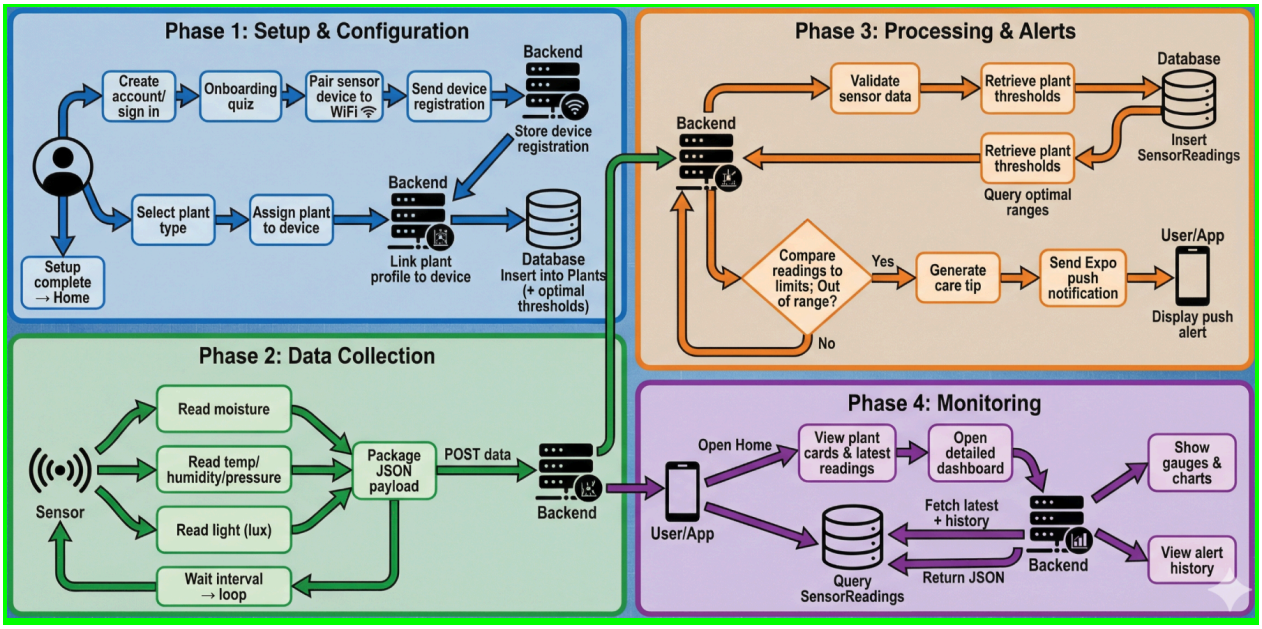

Figure 2: System Flow Activity Diagram [3].

3.3. User Setup

Users of the smart plant monitoring system must go through an initial setup process. The user first creates a personal account using the mobile app. This account will be used as the primary hub for organizing user-specific plants and will be where feedback notifications are sent to. Following account creation, the user will use the sensor device/hardware and attach it to the plant desired to be tracked. Then, the user pairs the sensor with the app via Bluetooth or WiFi. After connecting, the sensor will now be directly linked to the user's account. To customize it for the user profile, the user will use the app to select the plant type from a built-in database or manually configure care preferences. Once configured, the user and their plant are now in the system and will operate autonomously. The user may repeat the steps from attaching sensors to plants by configuring them in the app to add more plants for tracking.

3.4. User Experience

The user experience of the smart plant monitoring system is designed around one core principle: plant care should be effortless, not technical. The target users include beginner plant owners who may have no experience with the kind of plant monitoring technology required to gather environmental data. Every interaction should be designed with the intention of having the technology in the background, so the focus is on the plant.

After setup, the system should operate autonomously. The hardware collects sensor readings at regular intervals and transmits them to the backend over wifi. This happens all without any user input or action. Because the data flows through the internet, users will be able to monitor their plant from anywhere.

When a user opens the app, they will be presented with a card-style overview of all of their registered plants. Each card displays the plant name, a health status indicator, and the time of the most recent sensor reading. Tapping a card will open up a dashboard of that plant. Showing various readings like moisture, temperature, light exposure, etc. These readings will be presented through visual elements that are easier for beginner interpretation.

Below the indicators, there will be historical trend charts that show conditions have changed over time. Not only will it allow the users to identify patterns, but also allow the system to identify patterns and curate suggestions. The rest of the app interface will follow standard mobile navigation patterns like tab-based navigation and card-based lists. Errors, loading components, or empty components will be handled so the user is never confused about the system. The team want the app to feel as natural as checking the weather app.

If a sensor reading falls outside of the optimal range, then the user will be notified via push notification with a specific recommendation. For example, a user may be prompted, “Your plant could use water - soil moisture is low!” Critical conditions are immediately alerted, while more mild advisories are batched reasonably.

4. User Stories

The functionality of MatPotLib depends on the specific user needs identified by the team's brainstorming and requirement gathering sessions. To clearly define the needs, user stories have been created to represent the core pillars of the application.

4.1. Story 1: Onboarding

I am a tech savvy homeowner who uses smart devices often as part of my daily routine. I am beginning to get into indoor plant care, and want to use a smart plant pot to help me succeed. I value my time, so efficient onboarding is valuable.

- **Initial Setup:** As a first-time user, I want to pair my newly acquired plant pot with my phone through my home Wi-Fi. I'm familiar with this process because almost every other IOT device I have also uses this process.
- **Experience Tailoring:** I'm new to gardening, so during the onboarding quiz, I want to ensure I have options to specify my experience level, so recommendation levels match my comfort level.
- **Species Identification:** As a plant owner, I want to be able to input my own plant, even if I have some of the most exotic plants. I have some basic knowledge, and can do research on the plant care requirements, but I'm really excited for its automated data gathering aspect.

4.2. Story 2: Data Monitoring

I am a multi-tasking hobbyist gardener with many plants that I take care of. I have a large fleet of plants, and I just bought many plant pots to assist with keeping track of the health of all my plants. I need summaries of how my plants are doing every day, so I can make sure they receive top-notch care.

- **Quick Overview:** As a user with multiple indoor plants, I want to see a neat and easy to scan summary of all my plants' data on the home screen, so I can quickly identify any in need of care.
- **Detailed View:** When I select a specific plant, I want to see a detailed dashboard with historical and live numerical values for moisture, temperature, and light intensity.
- **Trend Identification:** As a frequent gardner, I want to view the 24 hour historical chart view on sunlight to see if my plant is receiving adequate sunlight throughout the day, especially since I am out most of the day for work.

4.3. Story 3: Proactive Notifications

I'm a busy professional with a very busy schedule. In order to help with my mental health, I decided to get a plant to care for, but I don't want to spend too much time caring for it and worrying about it. I'd like a solution that will tell me when I absolutely must water it, so I can take care of it and keep a good work-life balance.

- **Emergency Alerts:** As a busy professional, I want to receive a push notification when my plant's environment goes below a critical threshold, so I can water it before it begins wilting. I may potentially ignore the first couple of warnings.
- **Environmental Stability:** As someone who lives in place with unstable weather, I want to be notified if the temperature around my plant suddenly spikes or drops, so I can move the plant to a place with a more stable location.
- **Alert History:** I want to keep track of the care recommendations the app makes for me, so I can see how well I am doing at keeping my plants healthier over time. I expect many notifications in the beginning, but I hope it subsides over time.

5. Tech Stack

To support the operational workflow and user experience, the system relies on a multi-branch architecture. The system consists of a physical plant pot that has sensors, a microcontroller, and a battery, which is paired with a digital mobile application connected to a backend and database

pipeline. Sensor data is recorded at regular intervals by the hardware and is remotely transmitted to the backend service to be stored and analyzed. By comparing data with optimal conditions, the health status is determined, and recommendations are formulated into a notification sent to the user. Those are the requirements for the team's established Minimum Viable Product (MVP). Furthermore, future scalable additions can be integrated by adding new hardware components and introducing new frontend features powered by data collection and backend development.

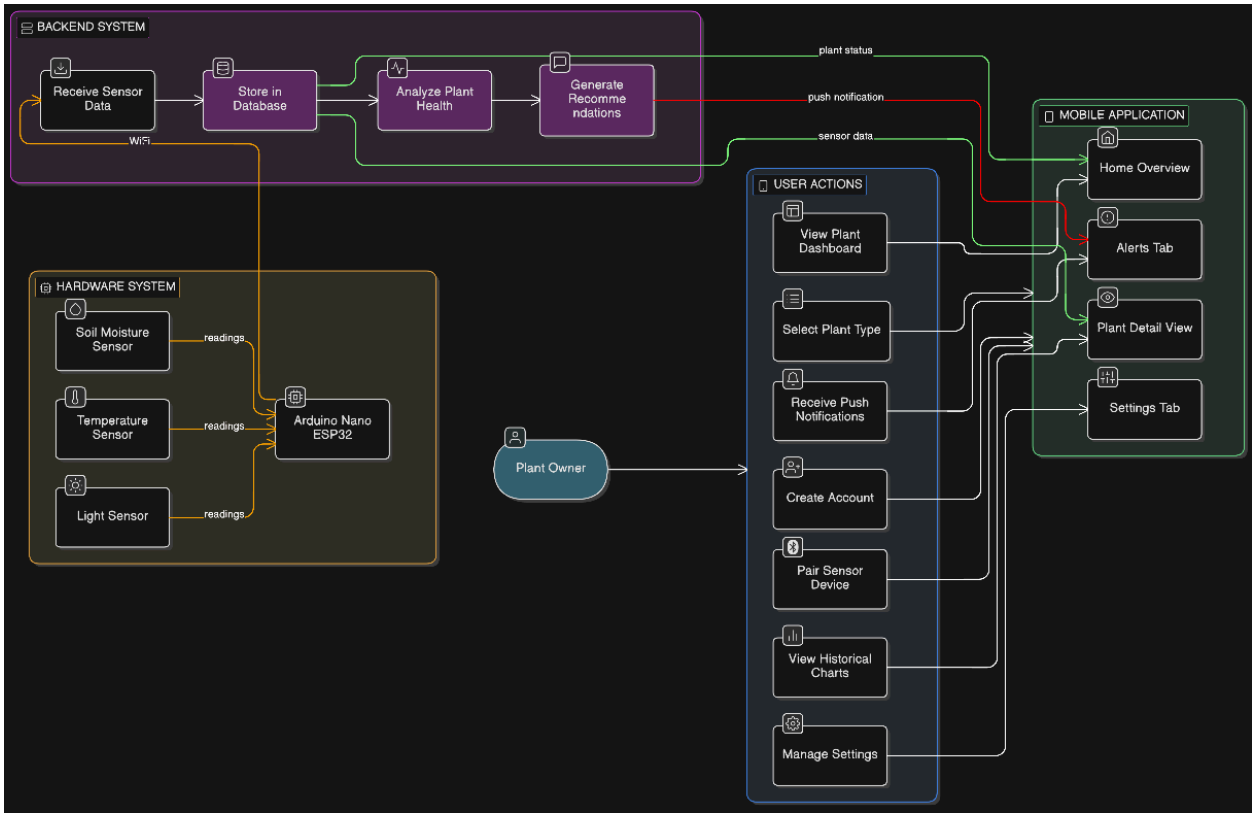

Figure 3: Use-Case Diagram [1]

5.1. Hardware

5.1.1. Component Selection

To support the system's functionality, a way to collect necessary data from the plant was needed to collect environmental recordings around the target plant and then send them to the system. The hardware to collect these recordings and to forward the data remotely would be a microcontroller with multiple unique sensors to be attached to the plant pot.

The main component of the microcontroller is the Arduino Nano ESP32, which is a compact, low-power-consuming piece with a built-in WiFi connection. This is responsible for periodically collecting data from the other pieces, reading the sensor values, processing the measurements, and transmitting the data over a wireless network to send to the backend system using WiFi. The Arduino Nano ESP32 was chosen for this project because it provides built-in WiFi connection, low power consumption, and compact size.

To monitor the soil conditions, a Capacitive soil moisture sensor was selected, which is a durable piece that records the moisture of the soil. A capacitive sensor was selected because they are durable and not as prone to corrosion. This will ensure that the product has longevity. This sensor is used to determine when the plant needs to be watered, track soil moisture levels, and help signal when to give watering alerts.

To capture the surrounding environmental conditions, the BME280 environmental sensor will be used to track the climate around the plant. This sensor reads humidity, temperature, and pressure of the surrounding air. These measurements allow the system to determine if the plant has a suitable environment.

Lastly, the BH1750 Lux sensor was chosen to measure the intensity of light received by the plant. This sensor provides digital light measurements by tracking daily light exposure and helping signal when to give low-light alerts. The light measurements are communicated to the microcontroller.

5.1.2. Hardware Integration

The hardware system will collectively include the ESP32 microcontroller, capacitive soil moisture sensor, BH1750 light sensor, BME280 environmental sensor, and the necessary power and breadboard components. Having all of the hardware components together like a single device ensures that the system can achieve the collection of environmental variables needed to monitor plant health.

The ESP32 microcontroller is mounted on the breadboard and properly wired to the power rails. The 3.3V and ground pins from the ESP32 are connected to the breadboard's power distribution rails, allowing power to be easily supplied to the sensors and any other components during development. Having that established is important in the hardware, as it enables easy integration and sensor testing with the system.

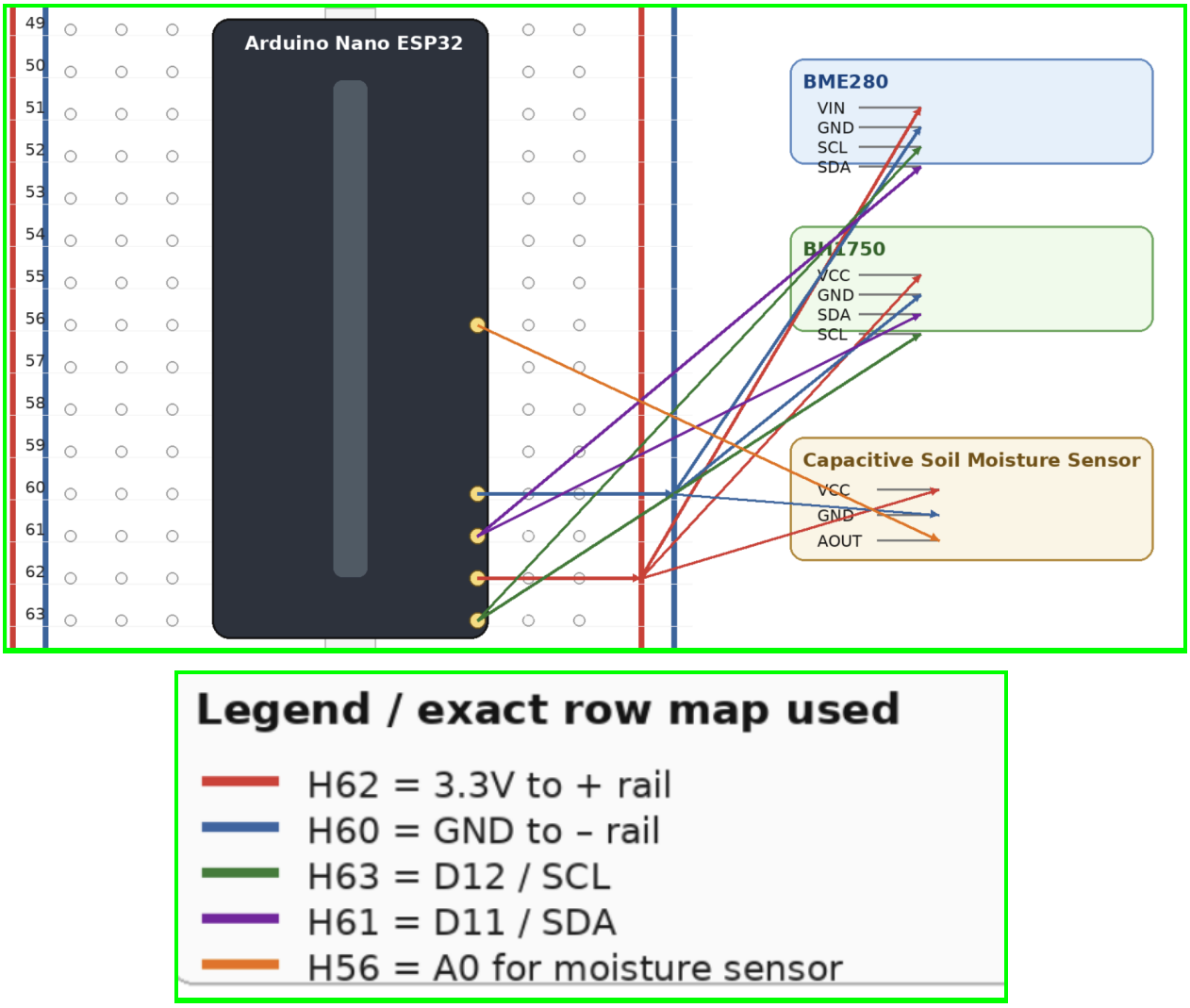

Figure 4: Diagram of Hardware Components

5.2. Backend

5.2.1. Framework Selection

The backend is built using NestJS with TypeScript, matching the frontend language (discussed later) and enabling consistent development patterns across the stack. NestJS provides a modular architecture that fits well with long-term scalability, and it naturally supports common backend needs such as authentication, validation, controllers, services, and dependency injection.

Other backend frameworks the team considered before selecting NestJS with Typescript included Express.js and Python-based frameworks. The team initially chose Express due to its popularity and the team's prior experience in the framework and with JavaScript. Express is a good foundational backend option that provides flexibility, but is less structured and can be difficult to manage and organize, causing scalability concerns. For Python frameworks, like Flask, the team considered them for their flexibility and robustness. They have a shallower learning curve than many other frameworks, but would still require around the same amount of learning as Nest because of the team's backend prior experience in JavaScript/TypeScript frameworks. This would also cause language inconsistencies across the stack, which the team wanted to avoid.

5.2.2. API Integration

The backend of the project will receive sensor readings from the hardware device, store them in a structured database, and call from the database to power the app features, like the dashboard, historical charts, plant configuration, and alerts. It serves as the central hub that connects the physical sensors and the mobile application together.

The backend comprises REST API endpoints used by the mobile application for tasks such as account management, plant creation, device pairing, viewing sensor data, and retrieving alerts.

For device communication, the backend includes endpoints specifically designed to process sensor data. The hardware device transmits readings at regular intervals, and the backend validates incoming payloads (device identity, timestamps, sensor ranges, and required fields)

before accepting and storing them. This validation step prevents corrupted or unrealistic data from polluting the database and improves overall reliability.

Endpoint	**Method**	**Description**
auth/signup	POST	Add new account with info
auth/login	POST	Log in account with info
auth/logout	POST	Log out of account with info
auth/forgot-password	POST	Send password reset link to email
auth/reset-password	POST	Reset user’s password w/ token
users/me	GET	Get account information
users/:me	PUT	Edit account information
users/quiz	POST	Submit quiz responses
devices/device	POST	Add new device
devices/devices	GET	Get all associated devices
devices/:deviceId	GET	Get device information
devices/:deviceId	PUT	Edit device
devices/:deviceId	DELETE	Delete device from user base
devices/:deviceId/pair	POST	Pair device with sensors via bluetooth
plants/plant	POST	Add a new plant
plants/plants	GET	Get all plants associated with user
plants/:plantId	GET	Get information on a plant profile

plants/:plantId	PUT	Edit a plant profile
plants/:plantId	DELETE	Delete a plant profile
sensors/readings	POST	Add new readings collected from device
sesnors/:sensorId	GET	Get sensor readings from specific device
alerts/:alertId	GET	Get a information on specific alert
alerts/notify	POST	Send a notification to associated user
alerts/notifications	GET	Get all active notifications

Figure 5: API Endpoint Table

5.2.3. Recommendations and Alerts

After each sensor reading is stored, the backend determines whether conditions are within healthy ranges by checking if values fall inside or outside safe thresholds. These thresholds are stored in another table that reflects a reliable dataset found online. After determining conditions, the backend generates a recommendation (ex. "Moisture has been low for several hours, consider watering soon"). This is sent as an alert entry tied to that plant and user.

Alerts are then delivered to the mobile app in two ways: stored in the Alerts table for history and visibility inside the Alerts tab, and sent through push notifications using Expo's push notification service, triggered by the backend after alert creation. This pipeline keeps user notifications consistent: every push notification corresponds to a saved alert record that the user can review later.

5.2.4. Authentication

All mobile API access is authenticated using JWT tokens, ensuring only authorized users can access their plants, devices, and sensor data. The backend does user ownership checks on API requests so that users cannot access data belonging to other accounts.

For device-specific endpoints, the backend uses a device identity mechanism (ex. device token or registered device ID) to ensure only valid devices can upload readings to the associated account. This prevents incorrect or unauthorized uploads, ensuring the reliability of the stored data.

5.2.5. Scalability

When developing the backend, scalability was considered since the scale of the database could grow massively and new features could be added to the system. Potential of new sensor fields, alert types, or tables could be made easily without disrupting the current workflow. Additional features could include advanced analytics, LLM tools, or machine learning algorithms. The core pipeline must remain the same to collect data, store, analyze, and deliver feedback to the user, while allowing more features to be added on top seamlessly.

5.3. Database

5.3.1. Database Selection

The data received from the hardware system must be stored and processed. Supabase was selected to store these records for future readings. By leveraging this framework, data can be easily hosted in a built-in PostgreSQL table database. This relational structure allows the system to manage a hierarchical system with organized data across multiple tables. Row-Level Security (RLS) is implemented to restrict access and only allow users to view and modify plant data with their own accounts through the API endpoints.

Supabase was chosen among several other backend data options. Firebase was considered because it had a built-in messaging service, which would reduce the number of dependencies, but it did not serve the project's need for relational data storage. MongoDB and other NOSQL alternatives were ruled out because they didn't support the highly structured nature of the data as well as a SQL database did. Supabase has proven to be a highly reliable and easy to integrate database, with many features alternative choices lacked, making it the clear winner overall.

5.3.2. High-Level Database System Architecture

To support multiple users and multiple plants per user, the system uses a relational database structure with clear ownership and data organization. The database is designed so data can be retrieved efficiently for both real-time views and historical charts. At a high level, the data model follows a natural hierarchy:

User → Devices → Plants → Sensor Readings → Alerts

Table Name	**Purpose/Content**
Users	User account information and authentication identifiers.
Devices	Represents each physical pot or sensor unit, including metadata like device ID, firmware version, last seen timestamp, and a reference to the owning user once paired.
Plants	plant profiles, such as plant name, plant type (optional), and the device associated with it. This allows the system to support multi-plant dashboards and clean organization in the UI.
Sensor Readings	Timestamped environmental data such as soil moisture, temperature, humidity (if included), and light intensity. This table is the largest-growing table and is optimized for time-based queries.
Alerts	stores notifications that were generated, including alert type, severity, message, and whether the user has viewed/dismissed it.

Figure 6: High-level table architecture

This structure supports the main product flow where readings arrive from the device, the backend stores them, evaluates them against plant conditions, and then logs and delivers alerts to the user. Each user may own multiple devices and plants, and each plant produces many

time-based sensor readings over the semester. Storing readings in a time-series style table allows efficient queries for charting and trend analysis.

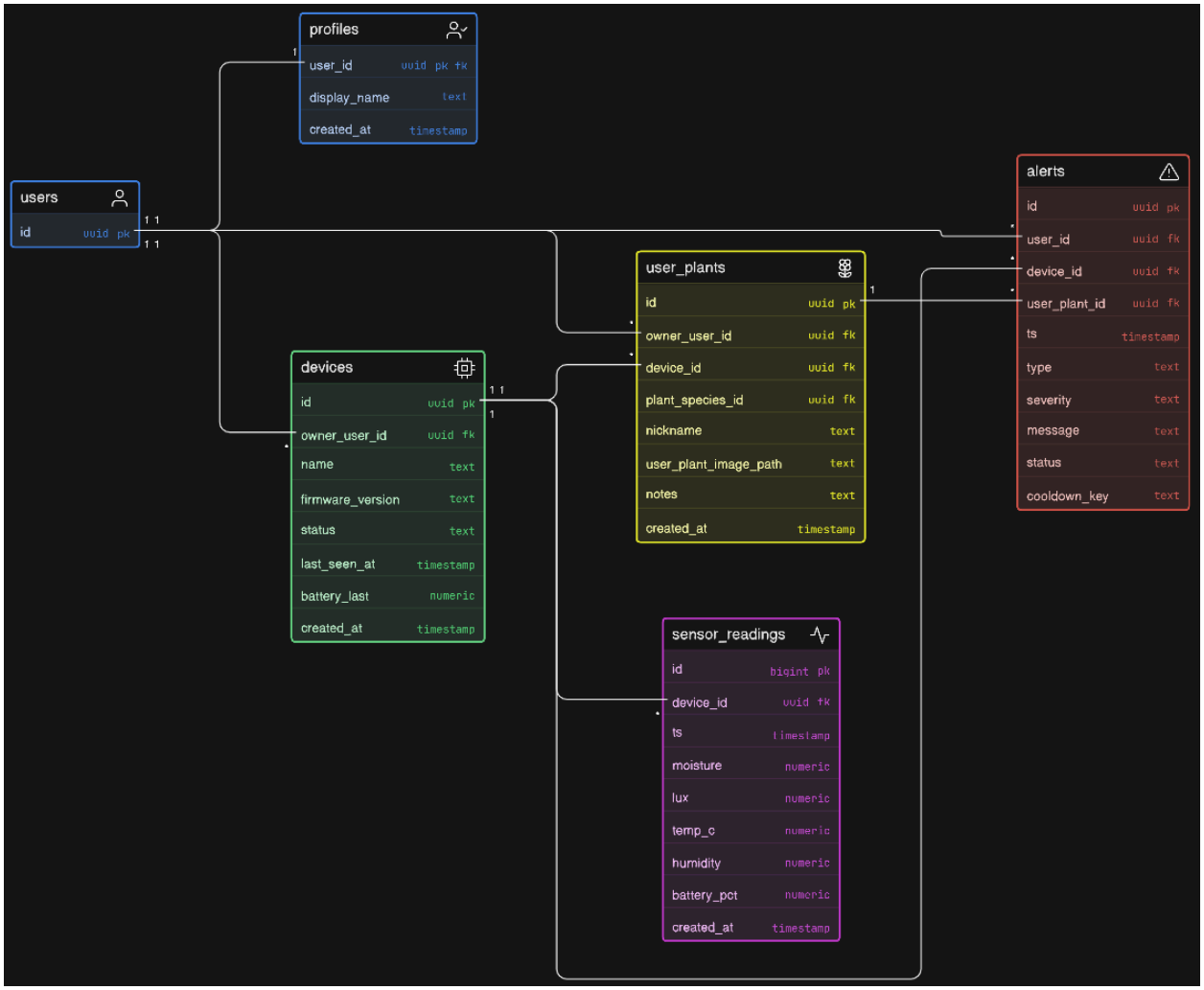

Figure 7: High-Level Table Hierarchy ERD [1]

The hierarchy shown in this ERD is important because it defines how data is structured, related, and accessed across the system. At the top level, the users table serves as the core entity, with each user uniquely identified by a user_id. This identifier acts as a primary key and is propagated throughout the database as a foreign key, enabling all related data to be linked back to a specific user.

The overall system architecture integrates multiple subsystems that work together to support user management, device monitoring, plant tracking, anomaly detection, and real-time alerting. The

design follows a modular and relational approach, where each subsystem is separated by responsibility but connected through shared identifiers such as user_id, device_id, and user_plant_id. This ID-driven hierarchy ensures modularity, reduces redundancy, and enables clear data flow from raw sensor inputs to user-facing insights and alerts.

5.3.3. Users and Profiles

At the foundation of the system is the users table, which represents all registered users. Each user acts as the central entity that owns devices, plants, and receives alerts. The profiles table extends user information, allowing additional metadata to be stored without modifying the core user structure.

In addition to the users table, the profiles table extends user information in a one-to-one relationship, while the devices table establishes a one-to-many relationship, allowing each user to own multiple devices via owner_user_id. These devices act as the primary source of environmental data and are directly connected to the sensor_readings table through device_id, forming a time-series relationship where each device can generate many readings over time.

Users table		
Field Name	**Field Type**	**Sample Data & Description**
id	String (UUID)	“a5f10pb4”, Unique identifier for individual user
email	Text (Unique, Not Null)	"user@example.com", Stores the user’s email address, which is used for authentication, communication, and account recovery. This field must be unique to prevent duplicate accounts and is often indexed to optimize login queries. Validation constraints ensure proper email formatting.

created_at	Timestamp (Default: CURRENT_TIMESTAMP)	"2026-03-01T12:34:56Z", Captures the exact date and time when the user account was created. This field is automatically populated by the database and is useful for auditing, analytics, and tracking user growth over time.
updated_at	Timestamp (Nullable)	Records the last time the user record was modified. This field is updated whenever changes are made to the user's data, supporting auditing and synchronization processes.

Figure 8: Users Table architecture

5.3.4. Devices

The devices table represents all IoT devices connected to the platform. Each device is linked to a user through owner_user_id, forming a one-to-many relationship. Devices continuously collect environmental data such as moisture, light, temperature, and humidity, which are stored in the sensor_readings table. This table acts as a time-series data store, enabling the system to track changes in environmental conditions over time.

Devices Table		
Field Name	**Field Type**	**Sample Data & Description**
id	String (UUID)	"dev_91ab23", Unique identifier assigned to each device. This ensures that every IoT device connected to the system can be individually tracked and referenced. It is used as a foreign key in the sensor_readings and alerts tables.

owner_user_id	UUID (Foreign Key → users.id, Not Null)	"a5f10pb4", Establishes ownership of the device by linking it to a specific user. This relationship enforces that each device must belong to a valid user, ensuring accountability and proper access control.
name	Text (Nullable)	“Living Room Sensor", A user-defined label for the device. This improves usability by allowing users to easily distinguish between multiple devices, especially in environments with several sensors deployed.
firmware_version	Text (Nullable)	"V1.2.0", Indicates the current firmware version running on the device. This is important for maintenance, debugging, and ensuring compatibility with backend services.
status	Text (Default: 'offline')	"Online", Represents the current operational state of the device. Typical values include 'online', 'offline', or 'inactive'. This field is updated based on device connectivity.
last_seen_at	Timestamp (Nullable)	"2026-03-05T08:22:11Z", Stores the last time the device communicated with the backend system. This is critical for monitoring device health and detecting connectivity issues.
battery_last	Numeric (Nullable)	87, Represents the most recent battery level reported by the device. This value helps users monitor device power status and plan maintenance or recharging.

Figure 9: Devices Table architecture

5.3.5. User Plants

User_Plants Table		
Field Name	**Field Type**	**Sample Data & Description**
id	UUID (Primary Key, Not Null)	"Plant_001", Unique identifier for each plant instance associated with a user. This allows multiple plants to be tracked independently within the system.
owner_user_id	UUID (Foreign Key → users.id, Not Null)	"a5f10pb4", Links the plant to its owner. This ensures that all plant data is associated with a valid user account and enforces access control rules.
device_id	UUID (Foreign Key → devices.id, Nullable)	"dev_91ab23", Associates the plant with a specific monitoring device. This relationship enables sensor data to be mapped directly to a plant.
plant_species_id	UUID (Foreign Key, Nullable)	"Species_rose", Identifies the species or type of plant. This allows the system to apply species-specific thresholds and recommendations for optimal plant care.
nickname	Text (Nullable)	"My Rose", A custom name given to the plant by the user. This enhances personalization and improves user experience when managing multiple plants.
user_plant_image_path	Text (Nullable)	"/images/plants/rose1.png", Stores the path or URL of the plant's image in cloud storage. This allows users to visually identify their

		plants within the application.
notes	Text (Nullable)	"Needs sunlight every morning", Allows users to store additional information or observations about the plant, such as watering schedules or growth notes.

Figure 10: User Plants Table architecture

The user_plants table acts as a bridge between users, devices, and plant-specific data. By referencing owner_user_id, device_id, and plant_species_id, it enables the system to associate a physical device with a specific plant owned by a user. This layered relationship is critical for contextualizing raw sensor data into meaningful plant health insights.

5.3.6. Sensor Readings

Sensor_Readings Table		
Field Name	**Field Type**	**Sample Data & Description**
id	BigInt (Primary Key, Auto Increment)	Unique identifier for each sensor reading. This ensures each data point can be individually referenced and indexed for efficient querying.
device_id	UUID (Foreign Key → device.id, Not Null)	Identifies the device that generated the sensor reading. This relationship allows readings to be grouped and analyzed per device.
ts	Timestamp (Not Null)	Represents the exact time the sensor data was recorded. This is critical for time-series analysis and trend tracking.
moisture	Numeric (Nullable)	Indicates the soil moisture level measured by the sensor. Used to determine watering needs.

lux	Numeric (Nullable)	Measures light intensity in lux. Helps determine whether the plant is receiving adequate sunlight.
temp_c	Numeric (Nullable)	Records ambient temperature in degrees Celsius. Important for maintaining optimal environmental conditions.
humidity	Numeric (Nullable)	Captures the surrounding air humidity level. Useful for plants sensitive to moisture in the air.
battery_pct	Numeric (Nullable)	Indicates the battery level of the device at the time of reading. Helps track device health.

Figure 11: Sensor Readings Table architecture

The sensor_readings table serves as perhaps the most important table in the system. It records measurements from the user's plant information to facilitate backend action. Each record is uniquely identified by an auto-incrementing ID (primary key) and mapped to the hardware from device_id, which is a foreign key referencing the devices table. This relationship enables the system to associate data points with a specific devices as well as to the user and plant it monitors. The ts (timestamp) field records the time that each reading was captured, enable the system with temporal analysis capabilities and trend tracking across hours, days, or weeks.

The table captures four environmental metrics: moisture, lux, temperature, and humidity. Each is stored as a numeric value. Soil moisture readings influence the system's watering recommendations, while lux values indicate whether a plant is receiving enough sunlight for its species. Temperature and humidity provide the context needed for plant care assessments. All four fields are nullable to account for instances when a sensor may be unavailable or malfunctioning, ensuring that partial readings are still recorded rather than discarded entirely. Finally, the additional yet vital battery_pct field tracks the device's battery level at the time of

each reading, allowing alerts for when the hardware needs recharging and to support device health in the long run.

5.3.7. Alerts

Alerts Table		
Field Name	**Field Type**	**Sample Data & Description**
id	String (UUID)	“a5f10pb4”, Unique identifier for individual user
user_id	UUID (Foreign Key → users.id, Not Null)	Identifies the user associated with the alert. Ensures alerts are delivered to the correct account.
device_id	UUID (Foreign Key → devices.id, Not Null)	Specifies which device triggered the alert.
user_plant_id	UUID (Foreign Key → user_plants.id, Nullable)	Description : Links the alert to a specific plant when applicable, allowing context-aware notifications.
ts	Timestamp (Not Null)	Indicates when the alert event occurred.
type	Text (Not Null)	Defines the category of alert, such as moisture_low or temperature_high.
severity	Text (Not Null)	Represents the urgency level of the alert (e.g., low, medium, high).
message	Text (Not Null)	Provides a human-readable explanation of the alert for the user.

status	Text (Default: 'active')	Tracks whether the alert is active, acknowledged, or resolved.
cooldown_key	Text (Nullable)	Used to prevent duplicate alerts within a defined time window by grouping similar alert events.

Figure 12: Alerts Table architecture

Finally, the alerts table integrates across multiple entities by referencing user_id, device_id, and user_plant_id. This allows alerts to be generated and traced back to the exact user, device, and plant involved. The use of IDs ensures referential integrity, efficient querying, and scalability, as all interactions between tables rely on consistent key-based relationships rather than duplicated data.

5.3.8. Devices System Architecture

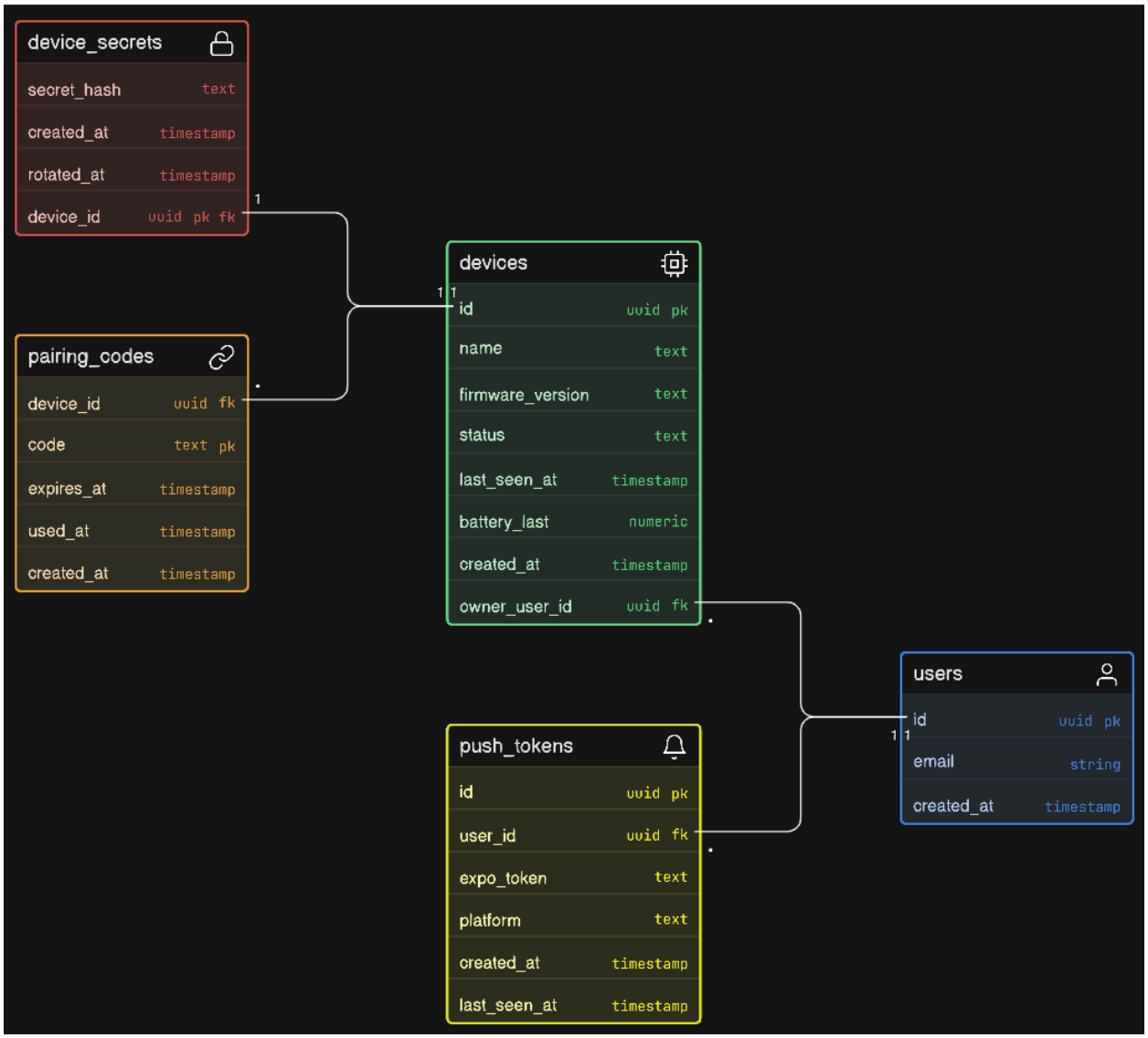

Figure 13: Devices ERD [1]

To ensure secure communication between devices and the backend, the system includes the device_secrets table, which manages authentication credentials for each device. The pairing_codes table supports secure onboarding by allowing devices to be linked to users through temporary, expirable codes. This ensures that only authorized devices can be registered. Additionally, the push_tokens table enables communication back to the user by storing device-specific notification tokens, allowing real-time alerts to be delivered to mobile devices.

5.3.9. Plant-Mapping System Architecture

Figure 14: Plant Species Dataset ERD [1]

The plant tracking subsystem is composed of the plant_species and user_plants tables. The plant_species table contains standardized data about different plant types, including optimal environmental ranges for moisture, light, temperature, and humidity. The user_plants table links this information to individual users and their devices, allowing each plant instance to inherit species-specific care requirements while also supporting user customization through nicknames, images, and notes.

5.3.10. Alerts and Events System Architecture

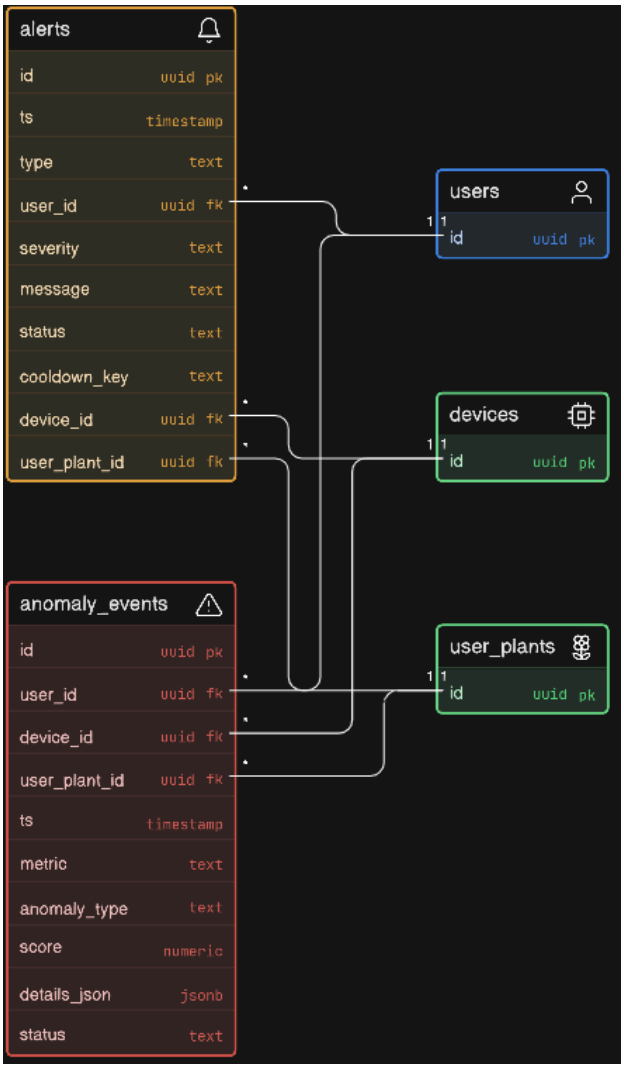

Figure 15: Alerts and Events ERD [1]

The anomaly detection subsystem processes sensor data to identify abnormal conditions. These events are stored in the anomaly_events table, where each record represents a detected issue such as low moisture or high temperature. Each anomaly is linked to a specific user, device, and plant, ensuring full traceability.

Building on anomaly detection, the alerting subsystem generates user-facing notifications through the alerts table. Alerts translate raw anomaly data into human-readable messages,

including severity levels and descriptions. The system also includes mechanisms such as cooldown_key to prevent duplicate notifications and improve user experience.

5.3.11. Data Flow

All subsystems are interconnected through foreign key relationships, ensuring referential integrity and enabling efficient querying across the entire system. Data flows through the system in a structured pipeline: devices collect sensor data, sensor readings are analyzed to detect anomalies, anomalies generate alerts, and alerts are delivered to users via push notifications.

Overall, this architecture separates concerns into distinct layers: user management, device management, plant data modeling, data collection, anomaly detection, and notification delivery. This modular design improves scalability, maintainability, and security while providing a clear and traceable flow of data from physical devices to actionable insights for users.

5.3.12. Query Strategy

The backend provides two main sensor data query paths: latest reading retrieval for the real-time dashboard (fast single-row query per plant), and time-window retrieval for historical charts (querying a range of readings by plant and timestamp).

To keep queries fast as the readings table grows, common query fields such as plant ID and timestamp will be indexed. For longer time windows, readings can also be downsampled in future iterations by aggregating values (for example, hourly or daily averages) to reduce payload size and chart rendering costs.

In addition to querying sensor data, users can also query plant data when selecting a plant. This table will be indexed on the plant common name and scientific name, allowing the user to use both to search for a specific species. If it doesn’t exist, the user can select an option specifying “other” where the backend will use OpenAI’s API to perform a websearch and add that enriched field to the database, where afterwards it will return it to the user. This new field will keep its

state for all users across the app. If the LLM fails to find details on the user's specified query, it will return none, and the user will see a message explaining that no information was found.

5.3.13. Data Quality

The data the team requires for this project consists of only the plant data used to verify if ideal plant conditions are being met. This Kaggle dataset was found and agreed upon as the primary dataset providing plant care information, though not limited to only that data. However, this dataset is not perfect, and lacks certain columns required, such as ideal moisture, humidity min and max, etc. Through exploratory data analysis, and using an LLM to go through the data and enrich missing columns, it was transformed to fit the required use case. Through random sampling of over twenty LLM-generated data points and verification, the team estimates that all the enriched data is accurate. As a fallback, if the user selects a plant not in the dataset, the product will use OpenAI's API for their GPT 5.4-nano model and web-search tools to generate a new row of data for the specified plant.

Through this process, it was found that the data planned to use is of high quality, and is sufficient enough for the use case. However, as new discoveries are made, the team is not adverse to modifying the dataset.

5.4. Frontend

5.4.1. Framework Selection

React Native with TypeScript was chosen as the mobile development framework. React Native enables the team to maintain a single codebase that compiles to native iOS and Android applications, which is essential given the team size and timeline. TypeScript provides static type checking that improves code reliability and aligns with the backend, which also uses TypeScript on NestJS, enabling consistent data models and interface definitions across the full stack.

Several alternatives were evaluated before selecting React Native. Flutter, developed by Google, was considered for its fast rendering engine and widget-based architecture. While Flutter offers strong performance and a growing ecosystem, it uses the Dart programming language, which no

team member had prior experience with and would introduce a language inconsistency with the TypeScript-based backend. Building natively with Swift for iOS and Kotlin for Android was also considered, as native development provides the best platform-specific performance and access to device APIs. However, this approach would require maintaining two separate codebases, effectively doubling the frontend workload for a team that only has one dedicated frontend developer. Given the project's timeline and team structure, this was not feasible.

React Native was ultimately selected because the team has prior experience with React's component-based architecture and JavaScript ecosystem, significantly reducing the learning curve. The Expo framework is used alongside React Native to simplify the build process, manage native module configuration through config plugins, and provide built-in services such as push notifications through Expo's push notification infrastructure. Expo also streamlines testing by allowing development builds to run on physical devices via the Expo Go app without requiring full native compilation during early development.

5.4.2. Library Selection

Through research conducted during the requirements gathering phase, core libraries were evaluated and selected for each major feature area of the application. For each category, multiple options were compared based on maintenance activity, cross-platform reliability, documentation quality, and compatibility with the Expo framework.

For data visualization, react-native-gifted-charts was selected to render sensor data on the plant dashboard. This library supports line, area, bar, and pie chart types with smooth animations and gradient fills, and is actively maintained with updates as recent as 2025. Two alternatives were evaluated: Victory Native and React Native Chart Kit. Victory Native is a well-regarded charting library with a declarative API, but its bundle size is larger and its React Native support has historically lagged behind its web counterpart. React Native Chart Kit was ruled out because it has not received a meaningful update in over three years and is considered deprecated by the community. react-native-gifted-charts provided the best combination of active maintenance, lightweight footprint, and support for the specific chart types needed to display moisture trends, temperature fluctuations, and light exposure over time.

For iconography throughout the application interface, lucide-react-native was selected as the icon library. Lucide provides a comprehensive set of over 1,500 open-source SVG icons with a consistent stroke-based design language that aligns with the application's clean, minimal aesthetic. Each icon renders as a scalable vector graphic, ensuring crisp display at any size across both iOS and Android without the rendering inconsistencies that occur with platform-dependent emoji. An alternative, react-native-vector-icons, was considered due to its popularity and large icon set spanning multiple icon families such as FontAwesome and Material Design. However, lucide-react-native was chosen for its lighter bundle size, tree-shakeable imports that only include icons actually used in the application, and its more cohesive visual style that avoids mixing design languages from different icon families.

For Bluetooth Low Energy communication during device pairing and WiFi provisioning, react-native-ble-plx was selected. This library provides a comprehensive API for device scanning, connection management, and GATT service and characteristic discovery, with support for both iOS and Android. An alternative, react-native-ble-manager, was evaluated and offers similar functionality, but react-native-ble-plx was chosen for its more extensive documentation, larger community, and its built-in Expo config plugin as of version 3.5.1, which simplifies native module integration without requiring a separate configuration package.

For push notifications, the expo-notifications library was selected in combination with Expo's built-in push notification service. This approach abstracts the platform-specific notification services, Apple Push Notification Service for iOS and Firebase Cloud Messaging for Android, into a single unified API. Third-party services such as OneSignal and Firebase direct integration were considered but determined to be unnecessary at the project's current scale. Expo's service provides sufficient functionality for registering device tokens, sending targeted notifications from the backend, and displaying native alerts on both platforms with minimal configuration overhead.

For navigation, React Navigation was chosen with a bottom tab navigator for the four primary screens and stack navigators for sub-flows such as onboarding and individual plant detail views. React Navigation is the most widely adopted navigation library in the React Native ecosystem

and provides well-documented patterns for the tab-based navigation structure required by the application.

5.4.3. Application Architecture

The application is organized around four primary navigation tabs that reflect the core user interactions: Home, Plant Data, Alerts, and Settings. The Home tab provides a multi-plant overview where each registered plant is displayed as a card-style summary showing the plant name, species, a health status indicator, the four most recent sensor readings (moisture, temperature, light, and humidity), and a timestamp of the last data update. The Plant Data tab presents a detailed dashboard for a selected plant, featuring gauge-style indicators for each sensor metric with visual in-range and out-of-range feedback, as well as historical trend charts rendered with react-native-gifted-charts over configurable time windows. The Alerts tab displays a chronological list of notifications generated by the backend, with severity-coded visual indicators distinguishing critical alerts, warnings, and informational messages. The Settings tab handles account management, device configuration, notification preferences, and logout functionality.

The application also includes an onboarding flow for new users. After account creation, the user is guided through a short personalization quiz that captures their gardening experience level, their primary goals for using the app (such as preventing overwatering or tracking plant health trends), and the types of plants they own or plan to monitor. These responses allow the system to tailor notification sensitivity and recommendation language to the individual user. Returning users bypass this flow and proceed directly to the Home screen after authentication.

State management is handled through React's built-in hooks and Context API, which provides sufficient complexity management for the application's scope without introducing additional dependencies such as Redux. API communication with the NestJS backend is performed over HTTPS using JWT tokens for authenticated requests. The application fetches sensor data from the backend via REST API calls using pull-to-refresh and periodic polling on the dashboard screen.

5.4.4. Design and Accessibility

A consistent design system was established to maintain visual coherence across all screens. The color palette is anchored by a nature-inspired deep green primary color (#1B5E2) paired with lighter green accents for interactive elements, a soft off-white background (#F5F9F5) for reduced eye strain, and white surface cards with subtle shadow elevation. Semantic colors are used for status communication: green (#16A34A) for healthy conditions, red (#DC2626) for critical alerts, amber (#F59E0B) for warnings, and blue (#3B82F6) for moisture-specific readings. These color pairings were checked to maintain a minimum contrast ratio of 4.5:1 against their respective backgrounds in accordance with WCAG 2.1 AA guidelines for text readability.

The interface relies on a standardized spacing scale (4, 8, 16, 24, 32 pixels) and a type scale ranging from 12 to 32 points, both defined as reusable constants to ensure uniformity. Sensor data is presented using visual indicators (gauges, color-coded bars, and emoji-based icons) rather than raw numerical values alone, making the information accessible to users with no technical background. All interactive elements are sized to meet minimum touch target dimensions of 44x44 points as recommended by Apple's Human Interface Guidelines and Google's Material Design specifications.

6. Agile Aspects

6.1. Methodology

We’ve adopted the fail fast and iterate quickly methodology that Agile teaches.

6.2. Meetings

6.2.1. Sprint Review Discussion

Sprint review meetings follow the same agenda, though it can vary slightly from sprint to sprint. The SCRUM master establishes the context of the team by revisiting sprint goals made in the previous meeting and discussing with each member what was and was not achieved in the goals

set in the previous cycle. From there, each member demonstrates part of their completed work, like a video of working parts or a walkthrough of new features added. All completed work is documented accordingly via Jira.

After the completed work is reviewed and assessed, incomplete work is addressed, and responsible members explain what blockers prevented that part of the sprint from being completed. These blockers will be acknowledged and considered in planning when carried over to the next sprint, where the next task is moved with a plan in mind. The SCRUM master also refers to the user stories implemented and determines if demands were met overall. Finally, ideas and feedback are shared amongst team members from the previous sprint, addressing questions like what went well and should be continued, what the team should work to improve on, and any priorities necessary to adjust before moving on to planning the next sprint.

6.2.2. Sprint Planning Discussion

The sprint planning meetings follow a similar process to the various meetings. Like the sprint review meeting, the spring planning meetings are also overseen by the SCRUM master. The meetings consider the review of the previous sprint, acknowledging all achieved in the previous sprint and covering tasks that were completed or not completed. All team members are asked how they are doing in terms of any issues they may be facing in their current workflow. After this review, the SCRUM master then works with the team to establish a goal that they will aim to accomplish in the upcoming sprint. The team sets goals for the layers of the project on all branches, including all of hardware, frontend, backend, and database, while also allocating tasks on documentation deliverables.

With this goal in mind, the SCRUM master will then review the backlog and any remaining tickets left over from the last sprint and pull in relevant tasks based on their priority level. The team will also discuss and come up with ideas for new tasks that are relevant to the goal of the sprint. The team will then collaborate to write user stories for all tickets going into the sprint to ensure that each task has a purpose. The team will then discuss their individual capacity for the upcoming two weeks, taking into consideration other coursework or personal obligations to ensure that everyone is not overwhelmed with tasks. The meeting will then end with time

estimation and task assignment. The SCRUM master and team will discuss and determine story points for tasks that do not already have them, and who will do the tasks based on roles, strengths, and availability for different focus areas. Work allocation is recorded through Jira, and all members are considered responsible for working on the assignment of and updating their own cards they are attached to within the next period.

6.3. Communication

6.3.1. Importance

Although the meetings serve as the primary, recurring form of contact to plan and discuss the team's work, communication between meetings is just as necessary to sustain the team's momentum. Problems and blockers may not occur on meeting days, and it is possible to lose valuable team time if a problem is not raised until the next meeting. Proactive communication provides a way to raise blockers, provide status updates, and ask questions in real-time without wasting time waiting for the next meeting or requiring all team members to be in the same physical location. To do this, the team has established various communication channels, each of which has different uses depending on the nature and necessity of the information being communicated.

6.3.2. Standards

The team expects every team member to be engaged and responsive across the team’s established means of communication, both in-person and remotely. (specified in section 2.2). All communication will be done respectfully and professionally, with no exceptions to disrespect toward a teammate. It is expected that there will be urgent updates throughout the duration of the project that will be important for all team members to hear and acknowledge. If a team member needs to interfere with another member’s part of the project, encounters a blocker, or, for any other reason, runs into conflict with another member, they are expected to communicate it promptly rather than struggling silently to prevent hindrance of the project in the long run.

6.3.3. Digital Communication Means

For a primary, everyday form of communication, the team decided to use iMessage groupchat. The team finds this platform to be the simplest way the team can talk to each other at any time, and where the team is the most responsive, since it goes directly to each team members' phones. This makes it easy not only for simple conversations, but for important, quick-notice alerts between each other, as well as for short-term discussions.

While iMessage does well with handling casual conversation and immediate alerts, the team's main hub for engineering work is on Discord. Unlike iMessage groupchats, Discord can be used on both mobile phones and desktop browsers; perhaps most prominently for technical work, servers can be divided into different channels for the organization of different project topics, making it a better host for the team's long-term technical discussions. For example, channels could be well-organized into general chat, ideas, and important links/documents, and separation of frontend/backend/hardware progress and debugging conversations. These channels also perform well at displaying/logging special messages - like code scripts and files - that can be easily shared and searched. Furthermore, Discord's voice chat is advantageous in teamwork because it allows voice calls on computers with active chatting and screen sharing. When the team decides to work with each other on the project code, Discord voice calls and chat will be where the team collaborates.

Jira serves as the team's project task and organization board, but also serves as a place for formal communication on project tasks/sprints. It serves mainly as a platform for organization, but is also used to share information about ideas, in-progress tasks, and completed sprints. By adding to or editing the board by writing new tasks or leaving comments on cards, the team formally communicates about progress.

6.4. Agile Development

The team has agreed to operate and develop under a SCRUM Agile framework, and all members are expected to participate in planning, maintaining, and completing sprints. These sprints will be strictly maintained with the team's Jira board, establishing tasks to be completed by and

discussed at future meetings. By completing project development in sprint-increments, it ensures consistent progress and involvement from all team members, and the project continues to receive deliverables and grow over the term. “Task-assigner” will be treated loosely, as all members can be open to adding sprints to the board, which feels proper for the state of the project, but is expected to be mainly done by the established Project Manager.

6.5. Engineering Documentation

By taking an Agile approach to engineering the project, completed sprints/checkpoints must be recorded for future purposes, in cases of reproduction or reference. For all branches of production from hardware, frontend, and backend, the team expects all work to be well-documented to ensure understanding of developed features when they are referred to for future production. Hardware parts should be described in a document in detail so its usage is made clear and the data it transfers to the software can be understood by the team’s software engineers. Code should be pushed regularly to the team’s shared GitHub repository with general commit titles and detailed descriptions to prevent cross-integration issues.

7. Jira

7.1. Usage

To maintain consistent progress toward the established MVP, the team utilizes Jira to plan and track sprints and epics. The team can manage and visualize the project's Agile workflow with this software.

7.2. Timeline

The team has broken down the project architecture into Epics based on progressive features that span throughout the production period of the semester. This timeline illustrates the established Jira roadmap in a Gantt chart-style visualization. Each epic is branched into sprints and tasks that dictate the schedule and completion of the team's project's features.

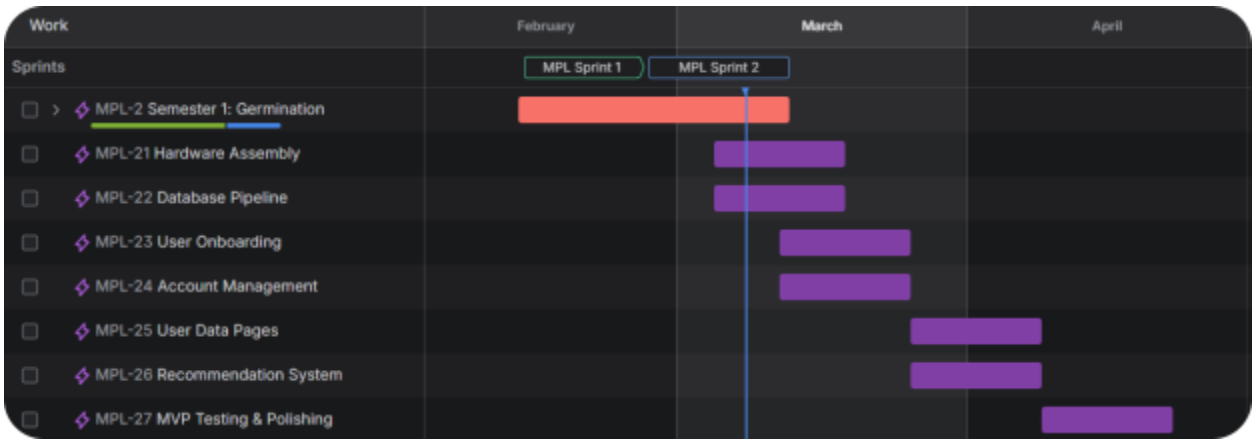

Figure 16: Jira Timeline for the project

7.3. Backlog

These epics are branched into many different tasks required to fulfill the requirements of that checkpoint. During the project planning and brainstorming, the team populated a backlog board with a draft of the necessary tasks to complete the team's project. These tasks have already been planned into scheduled sprints, as seen in the figure below. As the team proceed with sprints

within the team's development period, the team sorts the necessary tasks into the upcoming two-week sprints to further break down the general goals defined in the epics.

Figure 17: Jira task backlog

7.4. Sprint Board

When tasks from the backlog are scheduled for an upcoming sprint, they appear on the sprint-specific backlog board as well as the card board. These boards demonstrates active workflow, individual tasks and responsibilities, and live completion status. One of the team's sprints, "MPL Sprint 2" running from February 26th, 2026 to March 12, 2026 contains active and complete tasks, such as React application initialization, database setup, and documentation contributions. By assigning tasks as cards and tracking the status of each card, the team maintains accountability and ensures tangible progress in the product is delivered by the end of each cycle.

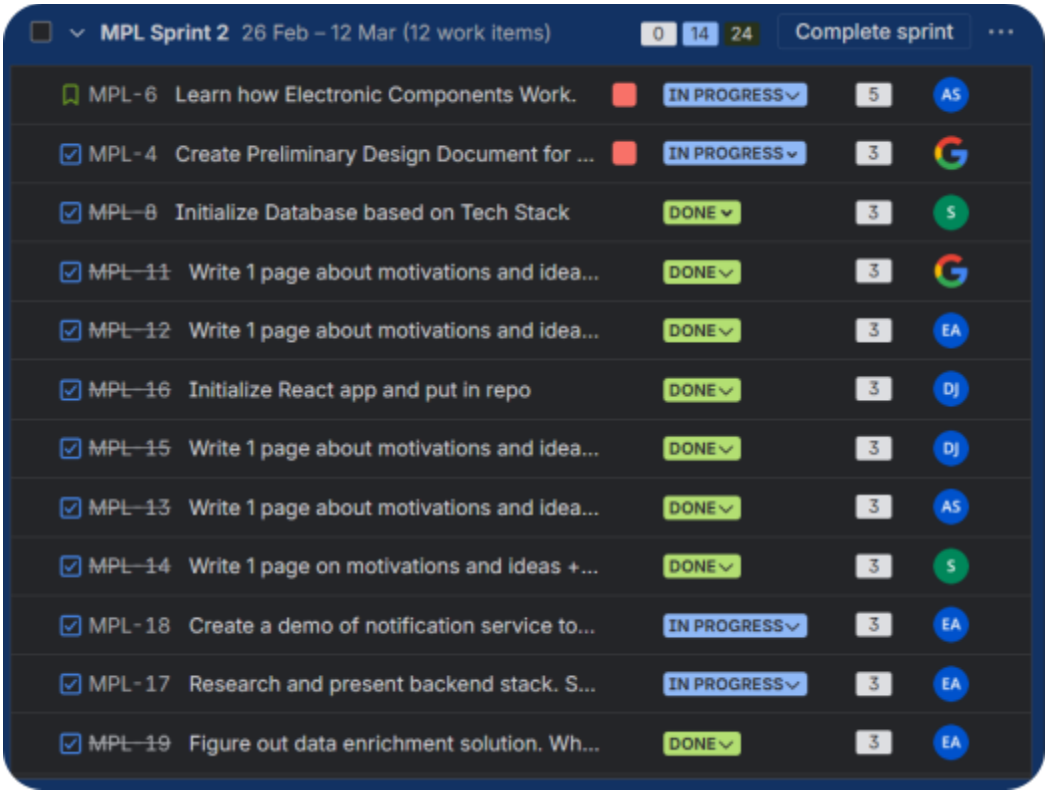

Figure 18: Jira Backlog for an individual sprint example

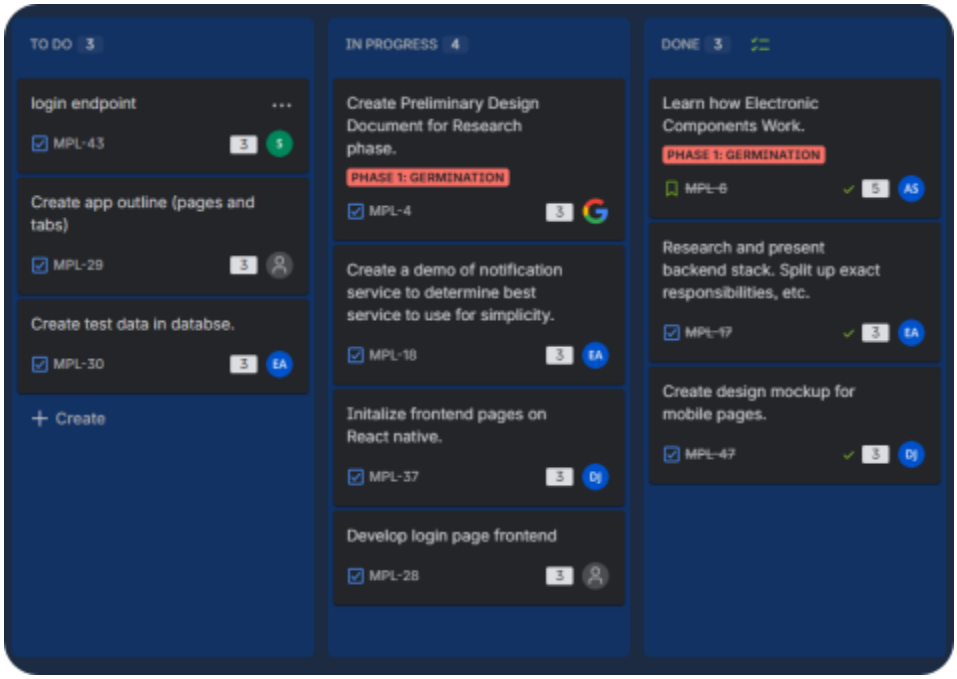

Figure 19: Jira Board for an individual sprint example

7.5. Proposed Sprints for Senior Design 1

The team is committed to continuing development of the project promptly following Agile methodology. To plan this, the team developed a plan for the team's semester-long work cycle divided into two-week sprints. Each sprint focuses on completing a small batch of progress and features across hardware, database, backend, and frontend branches. This approach ensures the team can produce tangible pieces of the product every two weeks, where the team can visually see progress. This model reduces the risk of falling short of the scope of the established MVP by telling us what should be done at each two-week checkpoint. The following table outlines the proposed sprint schedule for the semester, detailing dates of each sprint and tasks the team will work on within each cycle. Future tasks included in this table are also present in the list-form backlog, as shown in.

Sprint	**Date Range**	**Tasks**
MPL-S1	2/12 - 2/26	Define team roles. Determine meeting schedule, means of communication, and set up team GitHub and Jira. Learn tech stack and research electronics. Find a data source for plant condition reference.
MPL-S2	2/26 - 3/12	Initialize React Native frontend (packages), Node.js backend (packages), and Supabase database (tables and schema). Ensure frontend-backend-database is all connected, and interactions are tested. Finalize electronics selection and buy.
MPL-S3	3/12 - 3/26	Establish frontend styling to stay consistent with theme. Develop a login page and app outline (pages and tabs). Create test data in the database. Set up backend API calls to perform initial CRUD operations in the database. Code a call that can retrieve data from the plant-condition reference source (may store manually). Ensure that the login page and the home page can retrieve account-specific information. Assemble electronics and ensure

		they can collect and send data to the system.
MPL-S4	3/26 - 4/9	Complete the plant information tab, alerts tab, and settings tab in the frontend. Code API calls necessary for those pages. Develop a recommendation program to connect to alerts.
MPL-S5	4/9 - 4/23	Polish frontend design. Test system across hardware, database, backend, and frontend. Conduct user testing with a real user. Finalize a completed MVP.

Figure 20: Proposed Sprint Table for SD1

7.6. Proposed Sprints for Senior Design 2

The second semester of development focuses on refining the system to make it ready for presentation. With the basic MVP in place from the previous Senior Design 1, the team's sprint plan for the upcoming Senior Design 2 focuses on reassessing the incomplete work, extensive end-to-end testing, and reactively fixing issues when we identify them. With the shorter working term (due to presentation period), the team needs to be focused on a polished product, with each two-week sprint having specific goals to achieve. When possible, the team also wants to look into possibilities of expansion beyond the basic MVP, such as an AI-powered chatbot for plant care or manual/automated hardware such as an automated plant-waterer. The following table represents the proposed sprint schedule for the second semester, showing the dates and goals for each remaining sprint leading up to the final presentation.

Sprint	**Date Range**	**Tasks**
MPL-F1	8/24 - 9/7	Regroup team and assess the state of the MVP. Determine if there are any features left to implement or bugs left to fix from SD1. Prioritize remaining items in the backlog and update priorities in Jira. Start working on any gaps in functionality across branches. Determine the new focus for meetings and communication for the

		upcoming semester.
MPL-F2	9/7 - 9/21	Commence end-to-end testing for all layers of the system. Structured user testing sessions conducted feedback documented. Defects to be fixed reactively if identified. Consider possibilities for adding functionality beyond the MVP, like adding an AI chatbot for plant care or developing hardware functionality such as automated watering systems using sensor thresholds.
MPL-F3	9/21 - 10/5	All features of expansion should be finalized or locked if time constraints demand it. A final round of testing is conducted. The final presentation is created, which includes a live demonstration, project summary, and lessons learned. All documentation and code completed and ready for handoff.

Figure 21: *Proposed Sprint Table for SD2*

8. Mockups

8.1. Signup and Login

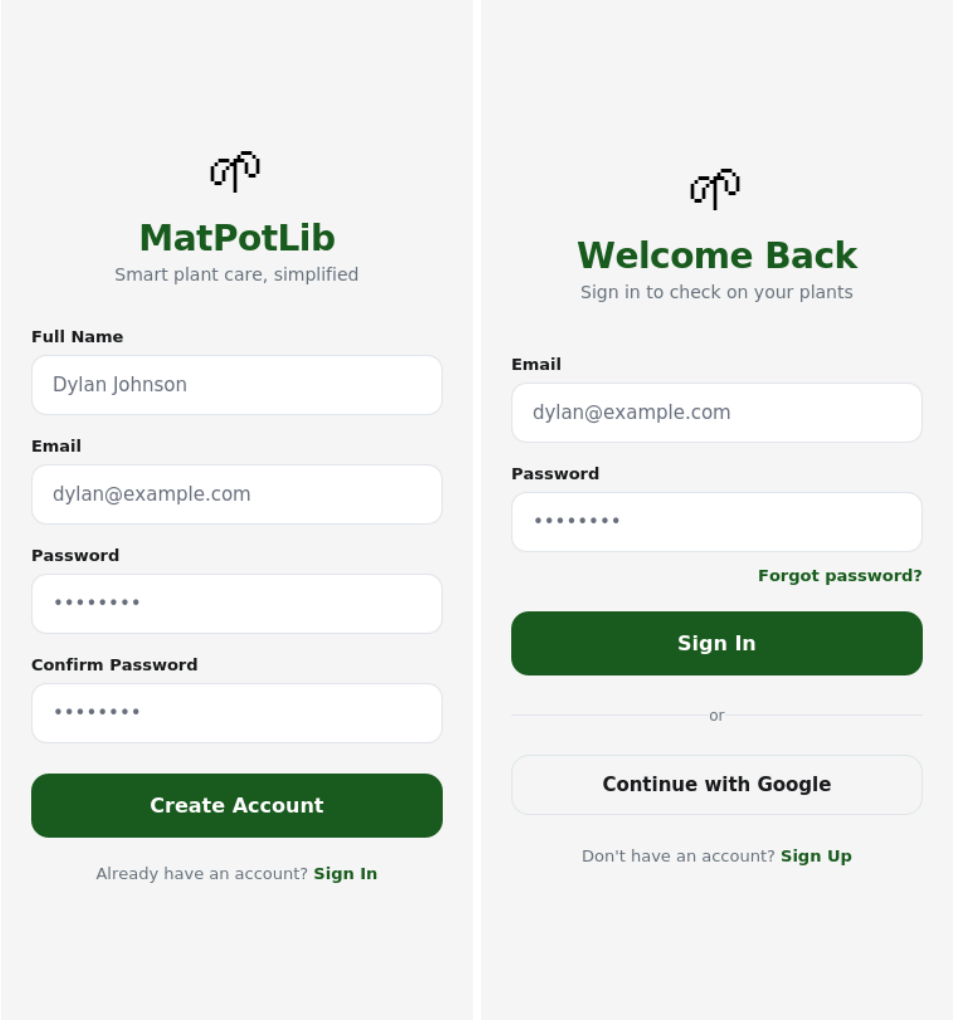

Figure 22: Signup and Login Page Mockups

The signup and login screens serve as the entry point to the application. New users are to create an account by providing their full name, email, and password. Returning users can sign in with their credentials or authenticate through Google. There is also have a "Forgot password?" link for account recovery. This minimal layout design with the MatPotLib branding prominently displayed ensures a straightforward experience to minimize complexity for users.

8.2. Onboarding Quiz

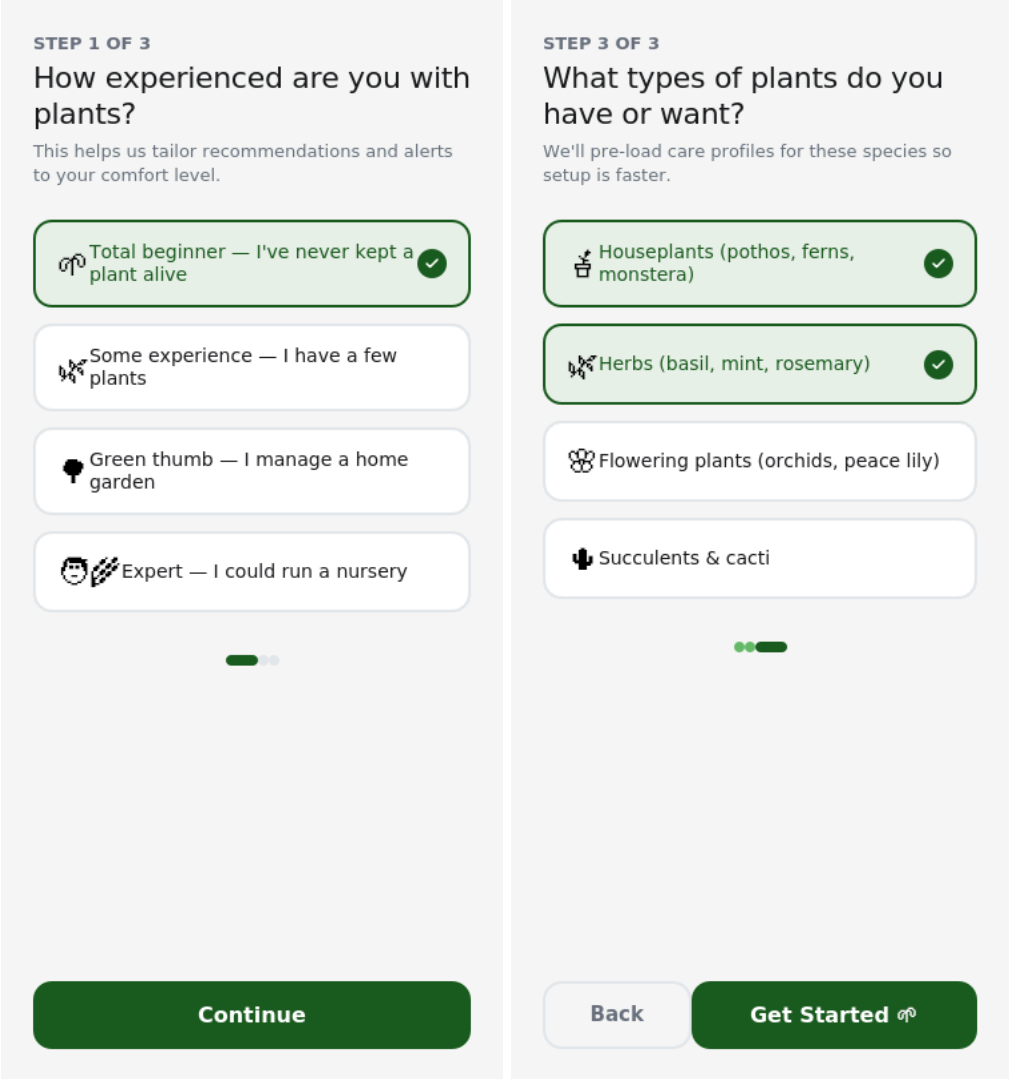

Figure 23: Onboarding Quiz Pages Mockups

When creating an account, users are prompted through a personalized experience via an interactive quiz with three steps. The user is first asked to provide their level of plant care experience, which can vary from a beginner to an expert. In the second step, they are prompted to choose the types of plants they have or will care for. This is done to provide personalized recommendations. This data is used by the recommendation engine to ensure that users receive relevant alerts from their first interaction.

8.3. Plant Monitoring

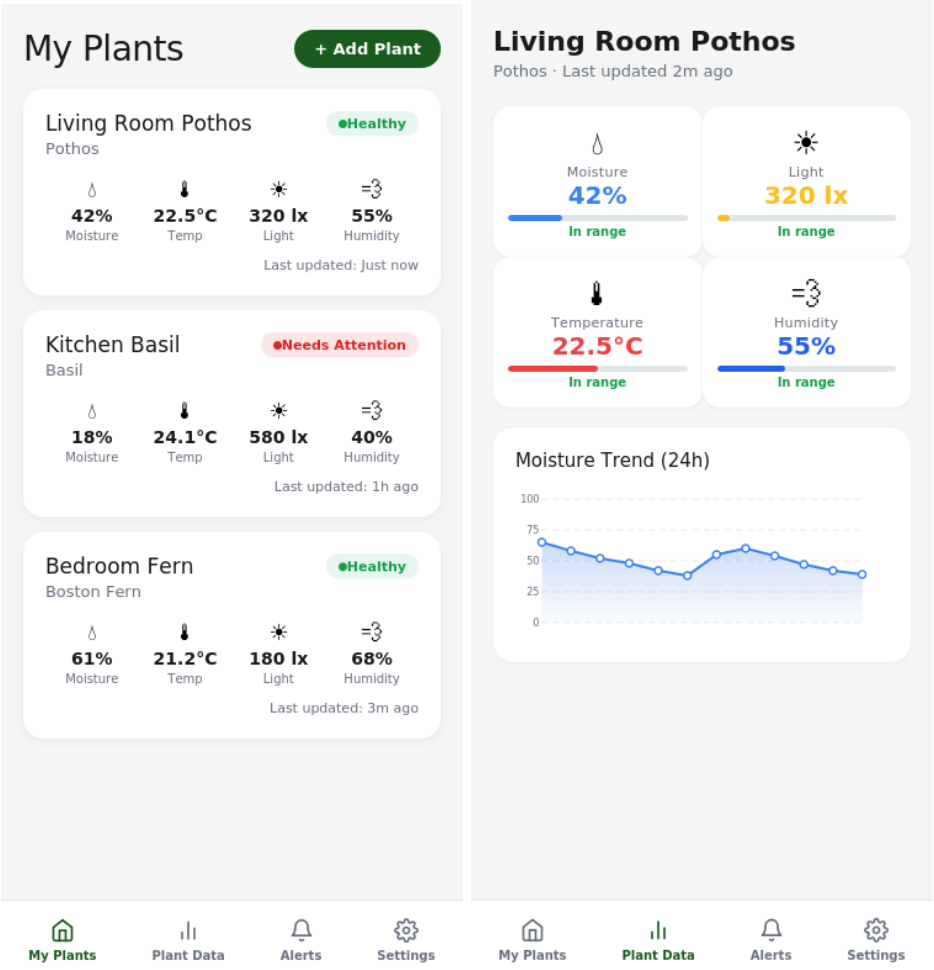

Figure 24: Plant Monitoring Pages Mockups

The plant monitoring screens are the central part of the application. The "My Plants" section allows for a quick overview of all plants, showing a summary of the current moisture, temperature, light, and humidity levels, along with a general plant health indicator in the top right for an at-a-glance report of the plant's conditions. When a user selects a specific plant, they are taken to a detailed view that shows a gauge bar for each of the plant's metrics, color-coded to quickly show whether the value falls within a safe range for that particular type of plant, as well as a 24-hour history chart for data-driven user analysis.

8.4. Alerts and Settings

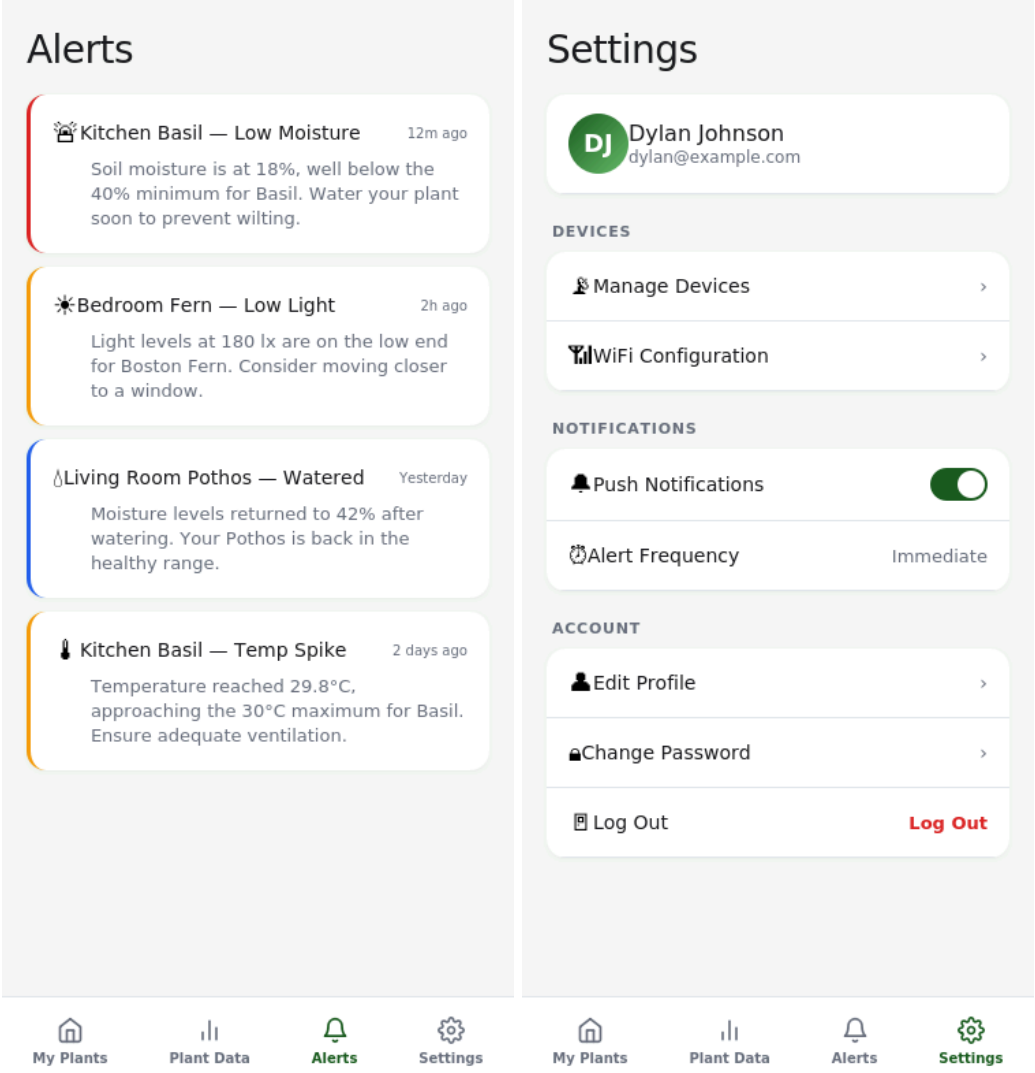

Figure 25: Alerts and Setting Pages Mockups

The alerts screen displays an ordered list of system alerts each color-coded based on its level of severity and along with a written description and suggested course of action. The settings screen organizes device management, Wi-Fi settings, notification settings, like push and alert frequency and account settings all on one simple screen. These screens collectively provide the user with complete control over the system's communication with them, whether they are more hands-on and want to be alerted immediately or more passive and want to be notified periodically.

References

[1]
“DiagramGPT – AI diagram generator created by Eraser,” *Eraser.io*, 2025. https://www.eraser.io/diagramgpt (accessed Mar. 29, 2026).
[2]
“API Platform,” *Openai.com*, 2025. https://openai.com/api/ (accessed Mar. 29, 2026).
[3]
“Nano Banana 2 - Gemini AI image generator & photo editor,” *Gemini*, 2026. https://gemini.google/overview/image-generation/ (accessed Mar. 29, 2026).